2027年热门会议推荐

具体咨询：白老师 13516196409

智驱产品技术领航

AI 时代产品和软件开发前沿系列峰会

核心定位

锚定AI 技术革新前沿，聚焦大模型与智能体工程化落地，解构 AI 时代软件工程架构革新路径，打通 AI 产品从技术突破到产品商业化验证的全链路。

关键词

- 大模型驱动
- 技术趋势
- Agent工程化
- 架构革新
- AI产品0-1
- 商业化验证

PM×AI 产品力领航者大会

2027年4月23-24·杭州

详情查看官网
演讲申报｜参会购票

AgenticAICon 智能体应用与架构工程大会

2027年6月18-19·上海

详情查看官网
演讲申报｜参会购票

AgenticAICon 智能体应用与架构工程大会

2027年11月5-6日·深圳

4月 · 5月 · 6月 · 9月 · 10月 · 11月

AI 赋能提质增效

QECon全球软件质量&效能大会

QECon 全球软件质量&效能大会

2027年5月21-22·深圳

QECon 全球软件质量&效能大会

2027年9月10-11日·上海

QECon 全球软件质量&效能大会

2027年10月15-16日·北京

演讲申报｜参会购票
详情查看官网

核心定位

聚焦 AI 技术在软件研发全流程的落地应用，以“质量保障&效能提升”为双驱动，助力研发团队实现软件研发的智能化转型。

关键词

- AI 赋能研发
- 效能提升
- 测试驱动
- 平台工具链
- 质量保障
- .....

生产级Agent设计蓝皮书·行业案例卷

"蓝皮书共建计划"启动——行业与场景专家 招募中

白皮书定义模式，蓝皮书记录落地。
把你的生产案例，写进行业参照系。
你的案例，很可能是这个行业第一份模式应用实践指南。

交流地点：《艾普·AI生产力研究院》展位咨询

扫码参与



共建计划

以真实案例为基础，持续组织行业与场景的 Agent 设计模式应用撰写。蓝皮书不写概念，只记录具名系统——它面对什么约束、踩过什么失败、为什么这样选架构、哪些经验可以迁移到你的行业。

我们招募这样的行业与场景专家

主导或深度参与过 Agent 系统建设，有真实跑在生产环境的案例；
在强监管、高可靠或大规模部署约束下做过工程取舍（金融、制造、政务、医疗、能源、电商等皆宜）；
愿意把一段经验，沉淀成可评审、可迁移的模式案例。

参与共建，你将获得

署名蓝皮书案例贡献者，成果进入 ADPS 公共技术资料长期存档；
用白皮书的模式语言系统梳理自己的实践，让团队经验成为行业参照；
参与共同体模块研讨与模式评审，共同界定下一版模式的边界；
与已在共同体的腾讯、蚂蚁、百度、京东等一线专家跨行业互审。

《企业级 Agent 设计白皮书》发布

纸质版 & 电子版 同步上线

领取地点:《艾普·AI生产力研究院》展位

Agent 正在从「会回答问题的模型」变成「能参与业务流程的系统」。模型仍是引擎,但决定它在生产中稳不稳的,是周围这套工程结构。本白皮书给出智能体工程的可复用架构词汇与评审标准。



27	7×6	3	5	6
矩阵模式规范	认知功能 × 执行拓扑	横切工程面 X1–X3	工程契约	步选型法

智能体工程的「GoF设计模式」

27个可复用Agent设计模式双轴框架

企业级智能体架构评审与选型实战手册

双轴框架

纵轴 · 认知功能

Agent 在做什么类型的工作

感知 Perception | 记忆 Memory | 推理 Reasoning | 行动 Action | 反思 Reflection | 协作 Collaboration | 治理 Governance

横轴 · 执行拓扑

数据与控制怎么流动

链式 · 顺序 | 路由 · 分流 | 并行 · 并发 | 编排 · 协调 | 循环 · 迭代 | 层级 · 分层

获取白皮书电子版

艾普·AI生产力研究院
Artificial Intelligence Institute of Productivity

Artificial Intelligence Institute of Productivity · 中立 · 开放 · 共创

面向 AI 原生软件工程的开放研究与实践组织

驾驭智能·定义 AI 时代新质生产力

深度承接《"人工智能+软件"专项行动实施方案》的专业化研究与交流平台

软件质量与效能 | **智能体工程与应用** | **六大研究方向** | **五大核心模块**

艾普 · AI 生产力研究院（AIIP）是一个中立、开放、共创的专业化研究与交流平台。研究院依托在软件质量与效能、智能体工程与应用等前沿领域的长期实践积累与前瞻研究底蕴，汇聚资深技术专家与企业研发团队等各方力量，共同推进行业能力体系建设、参考标准研究与评估框架研制，致力于成为支撑智能软件产业新质生产力发展的专业化交流平台。

更多详情请查看官网：https://aiip.org.cn/

关注官网

申请加入

与先行者同行，共赴 AI 生产力浪潮

六大研究方向

AI for SE
智能软件工程

AI赋能软件开发全流程、AI Coding 生产力与质量保障、智能测试工程，对标智能编程工具链与智能化技改。

度量体系
质效度量与治理

AI 质效度量模型、Token 成本治理、ROI 分析框架，以及成熟度评估参考框架的研究与共建。

ADPS
Agent 设计模式

Agentic 设计模式、多智能体架构、AI 系统技术栈与最佳实践沉淀，对标技能包与应用商店。

FDE
前线部署工程师

工程实践方法论、行业场景适配、企业落地案例与效能提升路径，对标就业友好与重点行业落地。

SE for AI
AI 工程化落地

AI 系统全生命周期工程化方法，可靠性与质量保障，以及 MLOps / LLMOps 交付与治理框架。

AI for X
AI 生产力

AI深度赋能千行百业，从软件研发走向办公、运营、数据、科研等真实业务价值场景，把工程能力转化为可度量的生产力。

AI 质效宣言

发起单位：QECon组委会 QECon Organizing Committee

从共识到行动，让宣言融入每一次研发实践

回归研发本质 **重塑人机协作** **推动行业共识**

协作共生 高于 局部取代
长效质量 高于 短期提效
适应进化 高于 固化流程
价值创造 高于 技术崇拜
数据驱动 高于 经验依赖
开放共享 高于 封闭自建

加入共建计划
获取电子画面

两层能力递进 · 一层工程底座

AI 增强软件研发全景架构

AI 嵌入 SDLC · 效能提升

• 人主导 • AI 协同

需求
- 行业调研与趋势分析
- 创意启发与功能探索
- 需求分析与规格细化
- 需求检查与内容评审

设计
- 代码理解与逻辑探测
- 技术选型与架构设计
- 接口设计与数据模型生成
- 技术方案评审与风险分析

开发
- 代码生成与缺陷解决
- 氛围编程与规约编程
- 单元测试生成与执行
- 代码评审与问题修复

测试
- 测试用例与数据生成
- 测试脚本修复与自愈
- 端到端测试生成与维护
- 缺陷发现与根因分析

CI / CD
- 构建与部署脚本生成
- 流水线失败诊断与修复
- 无人值守部署与发版
- 依赖升级与漏洞修复

运维 / 运营
- 告警聚合与摘要生成
- 故障分析与定位
- 故障处理与系统自愈
- 用户体验与反馈洞察

AI 研发范式变革 · 效能跃迁

• AI 主导 • 人监督

工程实践
- 驾驭工程 （模型决定上限，Harness 提升底线）
- 知识工程 （跨仓库代码与文档索引 · 沉淀复用）
- 验证闭环 （生成 → 验证 → 修正）
- 质量门禁 （构建多层次质量防护网）

流程重构
- 意图驱动、面向结果的交付 （替代多阶段接力）
- 对齐在前、执行在后 （人谈清边界，AI 自主实现）
- 需求即规格、证据即信任 （规格成为人机契约）
- 快速原型、持续迭代 （沿用敏捷 · 精益研发思想）

组织重塑
- 交付层抹平前后端分工 （跨栈交付 · 功能负责人）
- 架构师专注标准与门禁 （定标准 · 守门禁）
- 设计师与产品自主交付 （部分需求不经研发）
- 先锋队试点后规模复制 （先跑通再逐步推广）

效能度量
- 结果面：多 · 快 · 好 · 省
- 过程面：渗透率 · 成熟度 · 使用效果
- 数据源：代码 · 会话 · 配置 · 需求 · 监控
- 北极星：需求交付周期 · 单位 token 有效价值

AI 工程底座 · SE4AI （Software Engineering for AI）

• 自下而上 • 决定能力上限

模型层
- 多模型接入与路由 （模型网关 · 按能力与成本择优）
- 能力分级与降级 （强弱模型分级 · 故障回退）
- 领域微调与私有化 （微调训练 · 内网部署）
- 模型评测与选型 （基准跑分 · 场景实测对比）

Agent 层
- Agent 架构与编排 （单/多 Agent · 规划与控制流）
- 上下文工程与记忆 （上下文组装压缩 · 长短记忆）
- 运行时沙盒与工具接入 （隔离执行 · MCP 与技能调用）
- 灰度发布与回滚 （能力评估达标才放量 · 版本化 · 自动回退）

生态层
- 技能与智能体市场 （技能复用 · Agent 共享流通）
- MCP Hub 与工具生态 （工具目录 · 权限与调用治理）
- 知识资产与语料沉淀 （业务知识 · 代码知识 · 过程知识）
- 检索服务与知识图谱 （语义检索 · 图谱关系推理）

贯穿三层 | **安全与治理** 数据脱敏 · 提示注入防护 · 最小授权 · 输出合规 | **可观测性** 决策链路追踪 · 上下文溯源 · 失败归因 | **能力评估** 任务成功率 · 人工接管率 · 单任务成本

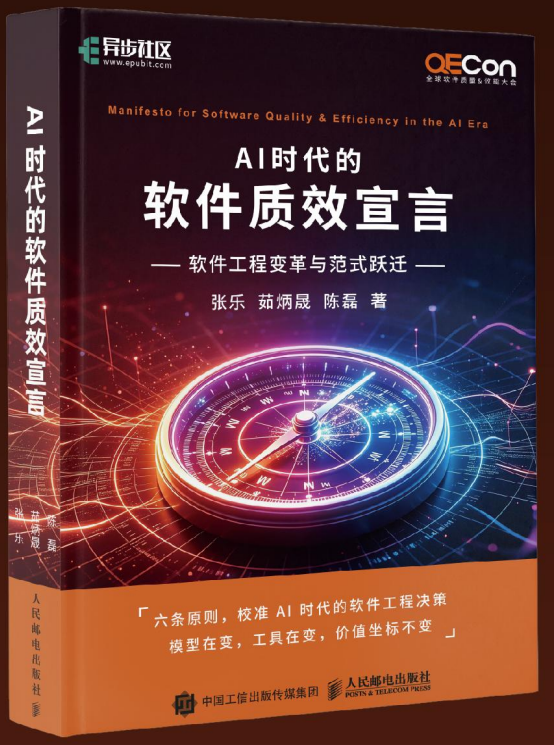
为每一位实践者提供一组稳定的价值坐标——当工具在变、模型在变、最佳实践在变的时候，这些原则帮你判断什么是不变的、什么是值得坚守的。

AI 时代的软件质效宣言

新书发布

— 软件工程变革与范式跃进 —

扫码订阅



系列电子书结构·三篇十九章 (道 · 术 · 势 —— 一条完整的 AI 研发变革路径)

道

第一篇 · 解读篇 (1-8 章)

确立价值观与原则：理解 AI 时代软件质效的本质，重建研发协作的信念底座，是全书的「道」。

术

即将发布

第二篇 · 单点提效篇 (9-14 章)

覆盖需求分析 → 架构设计 → 编码实现 → 测试验证 → CI/CD → 运维运营的关键环节，打通 AI 赋能研发全链路，是全书的「术」。

势

即将发布

第三篇 · 范式篇 (15-19 章)

当 AI 贯穿整条研发链路之后，组织形态本身将如何变革？从团队结构到协作范式，预见未来，是全书的「势」。

—— 主编 ——



张乐
腾讯
智能化软件工程
资深技术专家



茹炳晟
腾讯研究院
高级技术顾问



陈磊
前京东测试架构师

—— 作者 ——



王玉霞
北京兴云数科
过程改进专家



李佳奇
去哪儿旅行
技术总监



牛万鹏
百度
Comate研发经理



张晔
腾讯端服务质效
技术负责人



张涛
快手质量通道委员



付求爱
华为云
AIOps技术负责人



李萌
珊瑚智成
创始人&CEO



张刚
上海英慕科技
资深技术专家



张兆鑫
OPPO AI架构师



王磊
华为 AI技术专家

主办方：智盟创课

智盟创课（北京）科技有限公司（简称“智盟 Stu.”），作为专业IT产研培训服务供应商与AI时代数字化人才发展伙伴，为科技研发型企业及各类组织提供系统性、分层次、重实战的人才培养解决方案，同时搭建行业前沿技术交流平台，致力于成为企业数字化转型进程中人才培养与业务创新的核心支撑力量，推动组织实现成功转型及经营业绩持续增长。

同时，智盟发起并组织三大 IT 领域品牌会议：QECon 软件质量效能大会、SECon+AgentX 智能时代软件工程技术大会、PMxAI 产品力领航者大会、Agentic AICon 智能体应用与架构工程大会，助力产研中心深化数字化转型、聚焦业务价值达成，在产品研发的快速交付与高质量发展之间找到平衡，打通企业全维度学习视角，最终实现人才能力提升与企业竞争优势增强的双重目标。



详情咨询: 010-65798049

多元化学习形式

技术峰会 → 公开课 → 内训定制 → 微咨询 → 在线课程

核心培训方向

AI大模型应用 | 产品运营提升 | 数据分析决策 | 通用管理赋能 | AI职场综合应用

软件研发赋能

大模型前沿	架构设计	数据库	产品经理	持续运维	质量管理
移动开发	组织管理	云计算	开发编程	敏捷开发	大数据

Agentic AICon

AIO — 开发与使用分离的AI工程化平台实践

林明波 | 腾讯音乐 内容服务研发中心负责人



Agentic AICon

目录

CONTENTS

01 为什么要分离

02 整体架构总览

03 重点工程模块

04 总结与展望

背景：一个 Agent 平台，天然要满足三类诉求

开发者、运营、用户 —— 三类角色，同时向一个 Agent 平台提出诉求，方向完全不同

三类诉求
（角色画像）

同一个平台，三种角色站在完全不同的立场上提出要求：

开发者诉求

按需开发、持续调优 AI 能力

- 要的是灵活的工具链
- 能自由组合、替换底层能力

学习/投入意愿 高

运营诉求

拿到能直接提效的功能

- 不想关心底层怎么实现
- 看重的是「上线即见效」

学习/投入意愿 中

用户诉求

开箱即用

- 不想学习任何东西，只想要结果
- 决定去留的只是「好不好用」

学习/投入意愿 低

由此追问
（下一步）

三类诉求方向完全不同，如果都挤在同一条链路里满足，会发生什么？

这也是为什么「同一条链路」往往走不通——三类诉求需要被分层承接，而不是被同一套设计硬撑。

背景：错位带来的两个代价

诉求错位不会停留在「体验不好」这么简单——它会同时反映在产品和工程两个层面。

产品代价

用户诉求得不到满足，开发门槛挡住了真实需求

诉求错位 → 开发门槛太高 → 真实需求被挡在门外



- 用户要的是「结果」,却被迫先学「怎么做」
  - ·学习Skill安装和更新
  - ·学习配置MCP打通审核后台系统
  - ·学习配置定时调度任务
  - ·学习配置企业微信机器人结果通知
- 需求从「被满足」变成「被搁置」

真实表现：用户想要一键合并曲库同名歌手，却要先学会写 Skill——多数人最终选择放弃，退回手动运营，平台的能力等于白建。

工程代价

开发者与用户共用同一条 Agent 链路，链路复杂度层层堆高

开发者与用户共用同一条链路 → 链路复杂度上升 → 性能劣化



- 两类诉求交织在同一套设计里
  - ·开发在用户链路调试
  - ·用户在开发链路使用功能
- 复杂度沿着链路层层传导、难以隔离

真实表现：同一条链路既要保证终端用户生成MV视频的响应速度，又要扛开发者的高频调试请求——两者互相拖累，P99 延迟被迫上涨。

两者相互放大：产品端的门槛把用户逼向自建或放弃，工程端的负载又进一步拖慢响应——错位不是单点问题，而是会自我加剧的循环。

后面会有具体的问题与解决方案

先记住这个结论——工程代价不是抽象的说法，它会被拆解成可以量化的指标。

核心理念：A/B/C 三面架构与角色职责

开发与使用分离：A 面生产能力，B/C 面通过受控发布获得能力

A 面 • 开发者

内部研发 / 能力生产

按需开发、调试和持续调优

- Skill 开发
- 工具调试
- 效果调优
- 版本发布

受控发布 · 灰度开放

B 面 • 运营

内部 PM / 运营

日常工作提效

- 内嵌管理后台
- 业务工具即开即用

围绕运营任务组织操作流程

C 面 • 终端用户

外部用户

开箱即用，直接获得结果

- 多形态服务入口
- 无感知底层实现

围绕用户任务提供简洁体验

同一套能力底座

三面共享，边界清晰

共享底座能力

- 分别组织权限
- 分别组织功能入口
- 分别组织交互流程

让开发者构建与维护能力，让运营和用户直接使用服务

Agentic AICon

PART TWO

整体架构总览

02

AIO 平台整体架构

客户端层
（顶层）

开发客户端			运营客户端			用户客户端	更多客户端
租户1	租户2	.....	租户1	租户2	.....	Web/CLI/Android/iOS	.....

服务层
（中间层）

	A 面开发		B 面运营	C 面用户
安全稳定	网关鉴权模块	沙箱运行模块	Token认证模块	统一接入网关模块
功能开发	Skill开发/安装模块	任务编排模块	标准化接入	多端/协议适配模块
发布调试	灰度发布模块	远程调试模块	灰度策略	歌单制作
诊断管理	日志上报模块	可观测性模块	费用管理	音乐播放器
统计评估	数据统计模块	效果评估模块	业务数据看盘	歌曲打标签

业务底座
（支撑层）

数据处理引擎			内容匹配引擎			用户画像引擎			A/B Test	
检索	计算	融合	召回	匹配	推荐	采集	建模	标签	版本	分流

Agent底座
（智能层）

LLM引擎 （引擎兜底轮询增强）	**Chat Loop** （防死循环增强）	**Context Engineering** （上下文Token精简增强）
Memory （记忆摘要增强）	**Skills** （热加载增强）	**Tools** （安全增强）
MCP管理	**A2A Manager** （多Agent协作增强）	**Sessions** （多用户会话隔离增强）

业务底座
（治理层）

- 统一鉴权中心
- 用户权限中台
- 曲库版权后台
- .....

分层职责·三类客户端之下，三层能力各有明确职责

客户端按角色组织体验；服务层封装功能，业务底座承接决策，Agent 底座支撑通用执行

开发客户端·A 面

内部研发 / 开发者

按租户管理配置与 Skill，承接开发与调试

运营客户端·B 面

内部 PM / 运营

嵌入租户管理后台，业务工具即开即用

用户客户端·C 面

外部终端用户

Web / CLI / 小程序 / Android / iOS

服务层

面向角色的功能封装

将能力封装为角色功能，控制发布与使用

- Skill 开发/安装
- 灰度发布
- 远程调试

业务底座

面向场景的领域能力

沉淀可复用的业务计算与决策能力

- 用户画像
- 内容匹配
- 数据处理

Agent 底座

跨场景的通用运行机制

集中管理上下文、技能加载与执行产物

- Context Engineering
- Skills 热加载
- Resource 文件管理

各类入口复用底层能力，并按角色与租户暴露功能；图中纵向关系表示架构分层。

客户端体验按角色区分，业务逻辑持续演进，通用运行机制集中优化

Agentic AICon

PART TWO

重点工程模块

03

PART TWO

第一部分｜面向开发者

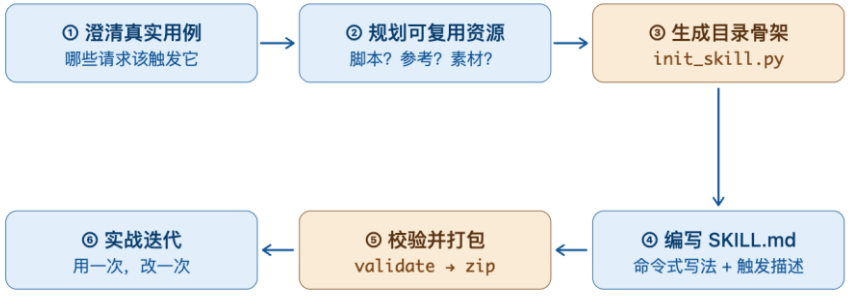
力交付的完整工程链路

Skill开发· 将业务能力封装为标准包

面向开发者 / 开发 → 安装 → 调试 → 灰度 → 观测回滚

A 面开发客户端 → Skill 构建与安装 → 调试验证 → 受控发布 → B/C 面按权限使用

skill-creator：用 Skill 生产 Skill



skill-creator中的3个核心脚本的作用

init_skill.py：负责建目录生成骨架

quick_validate.py：静态校验格式是否符合Skill规范

package_skill.py：校验+打ZIP包

标准 Skill 包：元数据、执行与验证契约

契约	字段 / 文件	运行时用途
能力标识	name / description / version	识别能力与管理版本
工具调用	tools / handler / parameters	工具名映射到实现函数
上下文注入	progressive_disclosure	选择注入章节与信息粒度
预算估算	token_estimates	参与运行时 Token 预算
执行与验证	entry.py / tests/ / examples/	执行、测试与使用示例

设计目标：5 分钟完成脚手架；生成后仍需校验、测试与调试。

开发阶段声明的契约，直接参与运行阶段的工具注册与上下文管理

统一安装：四种来源汇入运行时注册管线

来源适配集中在入口；SkillManager 管元数据，ToolRegistry 管工具调用

API 提交
POST /api/skills/install

本地目录
watchdog 监听用户目录

Git 仓库
clone → 临时目录 → 复制

系统内置
启动时扫描内置目录

SkillScanner：扫描目录 → 解析 SKILL.md frontmatter → 提取 metadata + tools

注册流程：文件变成可调用工具

01	校验	name 合规、tools 非空、handler 存在、依赖可用
02	动态加载	importlib 加载 entry.py，获得 handler 引用
03	同名更新	处理旧工具注册，更新 SkillManager 注册信息
04	发布到运行时	ToolRegistry 注册 source="skill" → 更新 _skills
05	通知	广播 skill_installed，供 SSE 推送

异常隔离与状态边界

单个 Skill 安装失败 → 记录错误

继续安装其他 Skill

动态加载无需重启整个服务

同名替换的部分注册失败

需要配套回退，try/except 不等于原子性

设计目标：从提交到 LLM 可调用 ≤ 30 秒

统一来源与注册契约，同时明确失败处理和版本替换的状态边界

远程调试：本地 IDE 进入真实执行现场

沿模型选择、工具分发与 handler 执行，定位环境相关问题

本地 IDE → SSH 端口转发 → 远程 debugpy → FastAPI 服务进程 → Skill handler

执行定位链：AgentEngine → ToolRegistry → Skill handler

调试过程



通过SSH将客户端调试器DAP和远程服务端debugpy连接起来，实现远程调试

沿一次调用拆解问题

模型侧	检查工具选择与参数
注册侧	名称查找、Schema 过滤、handler 映射
执行侧	环境变量、依赖返回、调用栈
复现侧	日志、追踪、Mock、录制回放

仅测试 / 预发启用：端口分配 + SSH 隧道 + 平台鉴权 + 操作审计；生产禁用调试入口。

端口隔离解决连接冲突；共享进程上的断点仍可能影响其他测试请求

灰度决策：稳定分桶控制新能力的可见范围

安装决定系统具备什么能力；灰度决定本次请求可以使用什么能力

配置规则：发布对象与准入条件

规则字段	控制语义
skill_id / version	目标能力与版本
allow_tenants / deny_tenants	租户范围
whitelist_users / blacklist_users	用户例外规则
gray_percent / default	放量比例与默认行为

复用于 Skill、功能开关、引擎版本与模型配置

稳定哈希：同一三元组进入同一个桶

```
key = f"{tenant_id}:{user_id}:{skill_id}".encode()
bucket = int(hashlib.md5(key).hexdigest(), 16) % 100
hit = bucket < gray_percent
```

桶号示例	5% 放量	20% 放量
3	命中	继续命中
17	不命中	新增命中

稳定性的边界

标识与规则共同决定

扩大百分比保留原有命中用户；小样本实际比例未必等于配置值。

version 不在哈希输入中；升级若改变 skill_id，将重新分桶。

名单、租户与百分比存在短路关系，规则优先级需要显式定义。

稳定分桶解决体验抖动，规则优先级决定实际开放边界

发布闭环：指标驱动回滚，配置传播决定生效时延

rollout_plan：5% → 20% → 50% → 100%

AgentEngine 执行 → CLS日志平台 → 指标聚合与判定 → 写入可观测平台 → 规则加载 / 告警

gray_decision_log 记录决策；触发阈值后写入 gray_percent=0

自动回滚阈值（文档配置）

指标	阈值	窗口
错误率	> 5%	5 分钟滑动窗口
P99 延迟	> 5000 ms	5 分钟滑动窗口
成功率	< 95%	5 分钟滑动窗口
用户投诉	> 10 条 / 分钟	实时

三类时间共同影响回滚响应

观察窗口 5 分钟
用多长时间的数据判定异常

计算周期 60 秒
聚合器多久计算一次

传播周期 30 秒
Apollo 规则多久重新加载

边界：停新增流量 ≠ 终止在途任务 ≠ 恢复旧版本；归零后仍需核对白名单等放行入口。

写入回滚配置，不等于所有请求瞬间恢复

PART TWO

第二部分 | Agent 底座

--支撑 AI 能力稳定、高效的运行

上下文工程：在有限窗口中保留更多决策信息

以四级压缩管线控制 Token 规模，同时提高相关信息占比

平均 Token	信噪比	关键信息召回率	成本系数
32K → 12K	35% → 78%	72% → 95%	1.0x → 0.4x
−62.5%	+43pp	+23pp	−60%

信噪比 = 与当前决策相关的信息量 ÷ 总 Token 数

① 截断
限制历史与单条长度

② 摘要
压缩早期对话

③ 渐进式披露
按预算注入 Skill

④ 输出截断
超长结果保留头尾

问题与目标：上下文占满不等于信息充分

问题 | 200 条聊天记录全部注入
窗口被历史噪音占用，当前决策信息被稀释

机制 | 压缩后只保留相关记录与记忆
四级管线先释放预算，再保留关键内容

结果 | 12K Token
覆盖更多决策信息

上下文工程同时管理规模与相关性

上下文筛选：动态预算减少无效 Token

预算决定注入深度，相关性决定保留内容，缓存降低重复计算

预算不足：按需降低注入深度

```
token_estimates
 entry_point: 320
 full: 1480
entry_point
 summary / when_to_use / quick_start
```

> 10,000	完整 Skill 文档
5,000–10,000	入口章节
< 5,000	入口摘要

历史过长：只保留当前相关信息

query Embedding → 余弦相似度

历史消息倒序评分，只保留 Top K（示例 100）

提取四类关键信息

- 实体
- 事实
- 偏好
- 决策

重复计算：过滤噪音并复用结果

问候语	直接丢弃
重复内容	语义去重，保留最新
早期结论	被覆盖后丢弃
失败重试	只保留最后一次
撤回调用	丢弃

Embedding → KMeans cluster_id

ctx_cache:{cluster_id}:{hash(query)}

相似 query 复用上下文构建结果

工具输出默认最多保留 10,000 字符，超限后头尾各保留 50%，中间标记省略字符数。

预算、相关性与缓存共同决定本轮上下文

Skills 热加载：不中断服务完成能力更新并控制风险

文件变化经过防抖、健康检查、灰度注册与自动回滚

三类目录监听 → 3000 ms 防抖 → 解析与依赖预加载 → 动态导入 → 健康检查 → 注册与灰度放量

不中断更新：监听三类目录

用户目录

~/.aio/skills/

系统目录

system_skills/

市场目录

marketplace/

发布前拦截：健康检查

元数据	必须包含 name 与 tools
工具定义	必须包含 name 与 handler
测试调用	使用 test_args，结果包含 success
失败处理	恢复旧版本，不向外抛出
审计	记录加载、卸载与回滚

影响面收敛：灰度、回滚与隔离

@skill_name	指定用户激活新能力
灰度放量	新版本逐步扩大可见范围
自动回滚	检查失败或运行异常时恢复旧版
异常隔离	单个 Skill 失败不影响其他能力
注入控制	去重并按 Token 预算选择内容

文档增强方案：三类目录监听、依赖预加载、健康检查、灰度发布、回滚与操作审计。

热加载提升迭代效率，灰度与回滚控制变更风险

AgentEngine：用有界循环防止工具任务失控或静默停止

统一编排主循环、SSE 事件、取消控制与后台进程

问题一：工具任务可能无限迭代

用户消息 @Skill 解析 → **LLM 推理** 流式输出 → **工具调用** ToolRegistry 执行 → **结果回填** 再次推理

最多 100 轮（可配） **文本、工具调用、工具结果 → 标准 SSE 事件流**

使用统计：对话次数、Skill 使用次数 → ~/.aio/stat/stats.json

问题二：取消或后台任务可能失去状态

同会话互斥	_active_sessions
取消标记	_cancel_flags，每轮开始前检查
过期取消保护	进入活跃状态后才应用取消
后台进程	未主动 poll 时由引擎兜底
轮询边界	每 3 秒，最多 300 次，约 15 分钟

长任务返回 process_id → **引擎持续识别活跃状态**

↓

遗漏轮询时自动 process_poll → **tool_stream 持续交付结果**

取消会话依赖循环检查；后台任务另由 ProcessManager 管理。

后台异步任务案例



AgentEngine 将推理、工具和状态控制收敛到有界循环

自推进与模型路由：恢复中断任务并选择可用模型

一侧修复中断的执行链路，另一侧选择满足能力与可用性要求的模型

静默停止：三层检查恢复执行

01 工具调用丢弃

tool_call_dropped：删除空 assistant 消息，直接重试

02 任务未完成

stop 但未完成判据命中：重新进入主循环

03 后台进程未结束

构造 process_poll，推送结果并推动 LLM 继续



生产案例：模型中断出发自推进

检查顺序：调用丢弃 → 任务未完成 → 后台进程 → 确认完成

模型不可用或成本过高：动态选择候选

用户显式指定	直接使用，不做二次路由
能力匹配	按任务类型筛选可达模型
健康度过滤	剔除近期失败率较高的模型
费用排序	按预计单次调用费用升序
失败降级	超时、报错或限流时取 fallback
队列耗尽	返回明确失败

类型	问题	路由结果	置信度	模型
简单	你好	✅ cheap	1.0	ep-c5e2eq9y
简单	1+1等于几	✅ cheap	1.0	ep-c5e2eq9y
简单	帮我翻译：Good morning	✅ cheap	1.0	ep-c5e2eq9y
复杂	请设计一个分布式缓存系统，需要考虑高可用、数据一致性和性能优化，给出详细架构方案	✅ expensive	1.0	ep-a4hg3lol
复杂	帮我分析这段代码的时间复杂度并给出优化方案：for i in range(n): for j in range(n)...	✅ expensive	0.8	ep-a4hg3lol
复杂	写一个完整的 Python FastAPI 项目，包含 JWT 认证、RBAC 权限、数据库 ORM、单元测试	✅ expensive	1.0	ep-a4hg3lol

实测：按任务复杂度进行模型路由

能力：推理 / 多模态 / Function Call　价格：可差 10–100 倍　稳定性：限流 / 超时 / Provider 故障

用户未指定模型时，候选顺序为能力满足、健康度可用、预计费用较低。

自推进负责继续执行，模型路由负责选择执行模型

PART TWO

第三部分 | 面向 B/C 端

--让能力可接入，让结果可使用

多端接入与租户隔离：快速落地业务并避免资源相互影响

统一三种接入方式，租户独立完成鉴权、配置与计算资源隔离

接入问题与目标

不同业务的交付形态不同

独立 Agent、已有应用和快速试用
需要不同接入路径

多个业务共享平台时

需要避免鉴权混用、资源抢占
以及配置相互影响

目标：灵活接入 + 独立边界

三种接入形态复用同一服务能力

1. **A2A 标准协议**
   独立 Agent 接入
2. **开放平台 API**
   应用按需组合模块
3. **官方页面**
   减少前端建设，快速落地

标准协议层和开放平台将接入差异收敛在服务层

租户接入流程与隔离边界

01 **租户与网关鉴权注册**

02 **A 面开发独立配置与 Skill**

03 **配置租户计算资源**

04 **按业务场景选择接入形式**

隔离目标

独立鉴权链路
计算资源与系统资源隔离
租户配置可个性化组合

统一接入降低落地成本，租户边界保障多业务稳定运行

Resource 管理：将执行产物交付为可管理的业务结果

FileTracker 统一捕获文件，元数据驱动预览、操作、隔离与清理

问题：文件只留下磁盘路径

Skill / Agent / 用户上传

图片、HTML、PDF、ZIP
音频、视频、文本

用户无法很好查找

文件来源与会话关系缺失

磁盘持续增长

HTML 预览存在脚本风险

机制一：FileTracker 建立统一追踪链路

1. **工具执行结果** — 自动发现生成文件
2. **FileMeta** — file_id / user_id / session_id
3. **存储与索引** — 类型 / 来源 / trace / 过期时间
4. **file_saved 事件** — 前端列表、预览与操作

机制二：让文件可用且可治理

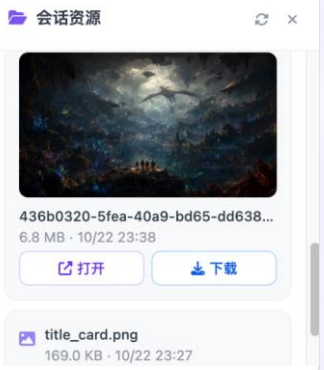
预览 图片浏览器；HTML iframe sandbox + CSP；PDF.js

操作 列表、筛选、下载、删除、重命名、分享

隔离 用户级 + 会话级目录，按 user_id / session_id 校验

清理 会话文件 30 天后通知并保留 7 天

保留 用户收藏永久；分享链接 7 天失效



元数据字段包括 file_id、session_id、user_id、文件类型、MIME、大小、来源、tool_call_id、trace_id、previewable 与 expires_at。

文件从执行副产物变成可管理、可交付的用户资源

Agentic AICon

PART TWO

总结与展望

04

总结：把能力建设留给开发者，把功能交给使用者

A/B/C 三面分离角色诉求，四段链路统一交付标准与风险控制

01 开发

A 面开发者

Skill 创建 / 安装
远程调试

02 发布

平台治理

稳定分桶
灰度放量 / 回滚

03 运行

Agent 底座

上下文工程
有界循环 / 自推进 / 路由

04 使用

B/C 面

资源管理
多端接入 / 租户隔离

贯穿各模块的四项设计原则

元数据驱动

Skill、灰度规则、FileMeta
把差异收敛为可配置数据

异常隔离

安装、热加载、工具与文件
局部失败不扩散到全局

渐进式控制

Skill 披露与灰度放量
按预算或风险逐级开放

全链路可恢复

健康检查、回滚、自推进
为失败建立明确恢复路径

分离角色入口，统一工程底座，才能让能力持续交付

展望：让平台持续优化，并在边界内复用更多业务能力

在可度量的基础上自动调整策略，并在隔离边界内扩展更多业务能力

评测闭环 01

用统一指标判断能力是否有效

- **推荐效果**：任务结果与用户反馈
- **运行延迟**：端到端耗时与工具等待
- **Token 开销**：请求、上下文与工具输出成本

策略与治理 02

运行数据驱动策略，并约束平台边界

- **模型与上下文**：按质量、成本和任务选择策略
- **灰度自动化**：指标异常时暂停或回滚
- **租户与资源**：配额、鉴权、隔离、清理与审计
- **接入治理**：A2A、开放 API 与官方页面

业务复用 03

在授权和隔离范围内复制能力

- **明确授权范围**：规定数据、Skill 与资源权限
- **保持隔离边界**：复用能力不突破租户边界
- **扩展领域能力**：沉淀可组合的业务模块

指标采集 → 策略评估 → 自动调整 → 结果复盘 → 反哺开发与运营

评测产生证据，策略持续优化，业务在隔离边界内复用

Agentic AICon

智能体应用与架构工程大会

感谢聆听

关注公众号

更多会议详情